

ML + Deep Learning

Essential Interview Questions

13 focused questions · No noise · Java + AI Engineer level · 2025

Contents

- Section 1 — ML Fundamentals (Q01–Q03)
- Supervised vs Unsupervised vs RL · Overfitting vs Underfitting · Train/Val/Test splits
- Section 2 — ML Algorithms Conceptual (Q04–Q11)
- Linear Regression · Decision Tree + Info Gain · Random Forest · Gradient Boosting + XGBoost
- K-Means + choosing K · KNN + drawbacks · PCA · Neural Networks + Hidden Layers
- Section 3 — Deep Learning Essentials (Q12–Q13)
- BERT vs GPT — Encoder vs Decoder · CNN — what it is and why it works

Section 1 — ML Fundamentals

Q01. What is the difference between supervised, unsupervised, and reinforcement learning?

Frequently Asked

Three fundamental ML paradigms — each suited to different problem types.

SUPERVISED LEARNING:

Model learns from labelled data — input-output pairs provided.

Algorithm finds the mapping: $f(X) \rightarrow Y$ where Y is known.

Examples: predicting house price (regression), spam detection (classification), image recognition, your RAG relevance scoring.

Real world: most production ML is supervised.

UNSUPERVISED LEARNING:

Model learns patterns from data with NO labels — just input X .

Finds hidden structure: clusters, patterns, representations.

Examples: customer segmentation (K-Means), document embeddings, anomaly detection.

Often used for data exploration or as preprocessing before supervised learning.

REINFORCEMENT LEARNING:

Agent learns by interacting with an environment and receiving rewards or penalties.

Trial and error: take action $\rightarrow$ get reward $\rightarrow$ update policy to maximise future rewards.

Examples: game playing (AlphaGo), robotics, recommendation systems optimisation.

Key connection to your work: RLHF (Reinforcement Learning from Human Feedback)

is used to align LLMs like GPT-4 and Claude.

Humans rate LLM outputs, model learns to prefer higher-rated responses.

 **Tip:** Connect RL to RLHF in AI Engineer interviews — shows you know why the models you use behave the way they do.

↳ Follow-up: What type of learning is RAG?

RAG itself is not ML training — it is an inference-time retrieval technique. The LLM inside RAG was trained with supervised learning (next-token prediction) and then aligned with RLHF. The embedding model for retrieval was trained with contrastive supervised learning — similar sentences should be close in vector space. RAG is an architecture pattern, not a learning paradigm.

Q02. What is overfitting and underfitting? How do you fix each?

Frequently Asked

Two fundamental failure modes of ML models.

OVERFITTING:

Model learns training data TOO well — memorises noise and specific examples.

Performs excellent on training data, poor on new unseen data.

Model is too complex relative to the amount of data.

Signs: training accuracy 98%, test accuracy 70% — large gap between the two.

Fixes for overfitting:

- More training data (most effective fix)
- Regularisation: L1 or L2 — penalise large weights
- Dropout: randomly zero out neurons during training
- Simplify the model: fewer layers, fewer parameters
- Early stopping: stop training when validation loss stops improving
- Data augmentation: artificially increase training data

UNDERFITTING:

Model is too simple to capture the pattern in the data.

Performs poorly on BOTH training AND test data — both scores are low.

Signs: training accuracy 62%, test accuracy 60% — small gap but both low.

Fixes for underfitting:

- More complex model: add layers, increase parameters
- Train longer: more epochs
- Better features: add more relevant features
- Reduce regularisation: too much penalty was constraining the model

★ **Key Insight:** Quick check: large gap between train and test accuracy = overfitting. Both scores low = underfitting. No gap and both high = well-fitted model.

Q03. What is a training set, validation set, and test set? Why three splits?

Frequently Asked

Three different subsets of data, each serving a distinct purpose.

TRAINING SET (typically 70-80%):

The data the model LEARNS from.

Model sees this data and optimises its loss function on it.

Weights and parameters are updated based on training set errors.

VALIDATION SET (typically 10-15%):

Used DURING development to tune hyperparameters and compare models.

Model does NOT train on this — but decisions are made based on it.

Example: "should I use 3 layers or 5 layers?" → check validation accuracy for each.

TEST SET (typically 10-15%):

Used ONLY ONCE — at the very end to report final performance.

Completely held out during all development. Never used to make any model decisions.

Represents truly unseen data — gives honest estimate of real-world performance.

WHY THREE SPLITS AND NOT TWO:

If you use the test set to choose between models, the test set effectively becomes a second validation set — your final evaluation is no longer honest.

The model has indirectly been tuned toward the test set.

Three splits maintains one truly unseen dataset for final unbiased evaluation.

💡 **Tip:** In interviews: if asked 'how do you evaluate your RAG system' — mention a golden test set of 100 Q&A pairs that was NEVER used to tune chunking, prompts, or retrieval. That is your test set. Shows you think about honest evaluation.

↳ Follow-up: What if you don't have enough data for three splits?

Use K-Fold cross-validation for the training and validation split. Split training data into K folds. Train on K-1 folds, validate on the remaining fold. Repeat K times — each fold serves as validation once. Average the K scores for a robust estimate. Still hold out a completely separate test set for final evaluation. Cross-validation replaces the fixed validation split, not the test split.

Section 2 — ML Algorithms (Conceptual)

Q04. How does linear regression work?

Frequently Asked

Linear regression predicts a continuous output by finding the best-fitting line through the training data.

THE MODEL:

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

y = predicted output (e.g. house price)

$x_1 \dots x_n$ = input features (area, number of rooms, location)

$w_1 \dots w_n$ = learned weights (how much each feature contributes)

b = bias (baseline prediction when all features are 0)

TRAINING:

Find the weights w that minimise Mean Squared Error on training data.

MSE = average of squared differences between predictions and actual values.

Two approaches: closed-form solution (small datasets) or gradient descent (large datasets).

ASSUMPTIONS:

Linear relationship between features and output.

Features are roughly independent.

Residuals (errors) are normally distributed.

WHEN IT WORKS WELL:

Linear relationship exists in the data.

Interpretability is needed — weight values show feature importance.

Useful as a fast baseline before trying complex models.

LIMITATIONS:

Cannot capture non-linear relationships.

Sensitive to outliers — they pull the line toward them.

↳ Follow-up: What is the difference between linear regression and logistic regression?

Linear regression predicts a CONTINUOUS value. Output is unbounded. Used for: price, temperature, sales volume. Logistic regression predicts a PROBABILITY (0 to 1) for BINARY classification. Applies sigmoid function to the linear output: $P(y=1) = 1 / (1 + e^{-z})$. Output bounded between 0 and 1 — interpreted as probability of the positive class. Used for: spam detection, fraud detection, medical diagnosis. Despite the name 'regression', logistic regression is a CLASSIFICATION algorithm.

Q05. What is a decision tree? What is information gain?

Frequently Asked

A decision tree makes predictions by asking a sequence of yes/no questions about the features, routing each example to a leaf node that contains the prediction.

STRUCTURE:

Root node: first split (most important question)

Internal nodes: subsequent questions

Leaf nodes: final predictions (class label or value)

Branches: the two possible answers to each question

Example for loan approval:

Root: Income > 50,000?

Yes → Credit Score > 700? → Yes: APPROVE. No: REJECT.

No → REJECT.

HOW IT DECIDES WHAT TO SPLIT ON:

At each node: tries every possible split (every feature, every threshold).

Picks the split that produces the most "pure" children — where one class dominates.

INFORMATION GAIN:

Measures how much a split reduces uncertainty.

Information Gain = Entropy(parent) - weighted average Entropy(children)

Entropy measures impurity: $H = -\sum p(\text{class}) \times \log_2(p(\text{class}))$

Pure node (all one class): Entropy = 0

Perfectly mixed 50/50 node: Entropy = 1 (maximum uncertainty)

The split with HIGHEST information gain is chosen at each node.

Gini Impurity is an alternative to entropy — similar results, slightly different formula.

PROS: interpretable, no feature scaling needed, handles mixed data types.

CONS: overfits easily (deep trees memorise training data), unstable with small data changes.

★ **Key Insight:** Decision trees are the building block for Random Forest and XGBoost. Understanding them is essential for understanding ensemble methods.

Q06. What is Random Forest? Why is it better than a single decision tree?

Frequently Asked

Random Forest is an ensemble method that trains many decision trees and combines their predictions — majority vote for classification, average for regression.

HOW IT WORKS:

1. Bootstrap sampling: create N different training datasets by randomly sampling WITH replacement.
Each dataset is same size as original, but some examples are repeated and some missing.
2. Feature randomness: at each split, consider only a RANDOM SUBSET of features.
Typical: $\sqrt{\text{total_features}}$ for classification tasks.
3. Train one full decision tree on each bootstrapped dataset independently.
4. Prediction: pass input through ALL trees, take majority vote (classification) or average (regression).

WHY BETTER THAN A SINGLE TREE:

Reduces variance (overfitting):

Individual deep trees overfit badly. But their errors are UNCORRELATED — different trees overfit to different noise. Averaging uncorrelated errors = errors cancel out.

Reduces sensitivity:

Single tree: change one training example, entire structure can change.

Forest: one example affects one tree out of 1000, overall prediction barely changes.

Feature importance:

Natural ranking of feature importance — how much each feature reduced impurity across all trees.

PRACTICAL STRENGTHS:

Works well out of the box with few hyperparameters to tune.

Handles missing values and outliers reasonably well.

Robust to irrelevant features.

Parallelisable — each tree trained independently.

WEAKNESS:

Not interpretable like a single tree — 1000 trees cannot be read.

Slower prediction than a single tree.

Does not extrapolate beyond training data range.

↳ Follow-up: What is the difference between bagging and boosting?

Bagging (Random Forest): trains N models in PARALLEL on different bootstrap samples. Predictions combined by averaging or voting. Goal: reduce variance. Models are independent. Boosting (XGBoost, LightGBM): trains models SEQUENTIALLY. Each new model focuses on examples previous models got wrong. Predictions combined by weighted sum. Goal: reduce bias. Models depend on each other. Key difference: Bagging = parallel, independent, reduces variance. Boosting = sequential, dependent, reduces bias.

Q07. What is gradient boosting? How does XGBoost differ from Random Forest?

Frequently Asked

Gradient Boosting builds an ensemble of weak learners (shallow trees) SEQUENTIALLY, where each new tree corrects the errors (residuals) of all previous trees combined.

HOW GRADIENT BOOSTING WORKS:

1. Start with a simple initial prediction (e.g. mean of all target values)
 2. Calculate residuals: how wrong is the current ensemble for each training example
 3. Train a new shallow tree to predict these residuals — the errors
 4. Add this tree to the ensemble with a small learning rate (shrinkage factor)
 5. New residuals = previous residuals minus new tree's predictions
 6. Repeat for N iterations — each tree is a small correction
- Together they build a powerful ensemble from many weak correctors.

XGBOOST vs RANDOM FOREST:

Random Forest:

Parallel training — trees are independent.

Deep trees — high variance reduced by averaging.

Reduces variance, prevents overfitting.

Less prone to overfitting with default settings.

Simpler to use with fewer hyperparameters.

XGBoost:

Sequential training — each tree corrects the previous.

Shallow trees — low variance base learners.

Reduces bias iteratively — each round gets better.

Can overfit with too many rounds, needs early stopping.

More hyperparameters: learning rate, max_depth, n_estimators, subsample.

Generally achieves better accuracy with proper tuning.

Built-in L1 and L2 regularisation on tree weights.

Handles missing values natively.

Won many Kaggle competitions — go-to for tabular data.

★ **Key Insight:** For tabular data: XGBoost or LightGBM is almost always the strongest baseline. Neural networks rarely beat them on structured tabular data with fewer than 1 million rows.

Q08. What is K-Means clustering? How do you choose K?

Moderately Asked

K-Means is an unsupervised algorithm that groups data into K clusters by iteratively assigning points to their nearest cluster centre and updating centres.

ALGORITHM:

1. Choose K (number of clusters)
2. Randomly initialise K cluster centroids
3. Assignment step: assign each point to its nearest centroid (minimum Euclidean distance)
4. Update step: move each centroid to the MEAN of all points assigned to it
5. Repeat steps 3-4 until centroids stop moving (convergence)

EXAMPLE USE CASES:

Customer segmentation: group customers by purchase behaviour.

Document clustering: group similar documents together.

Anomaly detection: points far from all centroids are potential anomalies.

LIMITATIONS:

Must specify K in advance — you might not know the right value.

Sensitive to initial centroid placement — can converge to poor local minima.

Assumes spherical clusters of similar size — fails on elongated or irregular clusters.

Sensitive to outliers — they drag centroids toward them.

Only works with numerical data directly.

HOW TO CHOOSE K:

Elbow Method:

Plot WCSS (Within Cluster Sum of Squares) vs K.

WCSS always decreases as K increases — more clusters means points are closer to centroids.

Look for the elbow: where adding another cluster gives diminishing returns.

The K at the elbow is the optimal trade-off between fit and simplicity.

Silhouette Score:

For each point: measure how similar it is to its own cluster vs nearest other cluster.

Score range: -1 to 1. Higher is better defined clusters.

Choose K with highest average silhouette score.

Domain Knowledge:

Often the best choice — if you know you want 5 customer segments, use K=5.

💡 **Tip:** K-Means++ initialisation (default in sklearn) is better than random initialisation — spreads initial centroids far apart, reducing chance of poor convergence.

Q09. What is KNN? What are its drawbacks at scale?

Moderately Asked

KNN (K-Nearest Neighbours) makes predictions by finding the K most similar training examples and using their labels.

HOW KNN WORKS:

For Classification:

1. Given a new point, find the K training examples closest to it by Euclidean distance.
2. Take majority vote among their labels.
3. Assign that label to the new point.

For Regression:

1. Find K nearest neighbours.
2. Predict the AVERAGE of their target values.

CHOOSING K:

Small K (K=1): very flexible, overfits to noise, high variance.

Large K (K=100): smooth predictions, may underfit, misses local patterns.

Optimal K: typically odd to avoid ties, found via cross-validation.

CHARACTERISTICS:

No training phase — all training data stored in memory, learning happens at prediction time.

Non-parametric — no assumptions about data distribution.

Feature scaling is MANDATORY: distance is affected by scale. Normalise first.

DRAWBACKS AT SCALE:

Slow prediction: $O(n)$ — for each new prediction, compute distance to ALL n training examples.

With 1 million training examples: 1 million distance calculations per prediction.

Memory: stores entire training dataset. Cannot scale to very large datasets.

Curse of dimensionality: in high-dimensional spaces, all points become equally distant.

Distance metrics lose meaning above roughly 20-30 features. KNN breaks down.

Sensitive to irrelevant features: irrelevant features add noise to distance calculation.

Feature selection is essential before applying KNN.

↳ Follow-up: How does KNN relate to vector database retrieval in RAG?

RAG retrieval is essentially KNN search in embedding space. Given a query embedding, you find the K nearest document embeddings by cosine similarity. The key difference: vector databases use Approximate Nearest Neighbour algorithms like HNSW instead of exact KNN. HNSW builds a graph structure enabling sub-linear search time — finding approximate nearest neighbours in milliseconds even across millions of vectors. This is why Qdrant and Pinecone can do fast retrieval at scale while plain KNN cannot.

Q10. What is PCA and when would you use it?

Moderately Asked

PCA (Principal Component Analysis) is an unsupervised dimensionality reduction technique that transforms high-dimensional data into a lower-dimensional representation while preserving as much variance (information) as possible.

WHAT IT DOES:

Finds the directions of maximum variance in the data — called principal components.

Projects the data onto these directions.

First component: direction of highest variance.

Second: direction of second highest variance, perpendicular to first.

You keep only the top K components — much smaller than original dimensions.

INTUITION:

If data exists in 100 dimensions but lies mostly on a 2D surface,

PCA finds that 2D surface and gives you 2D coordinates.

You go from 100 numbers per example to 2 numbers per example.

WHEN TO USE PCA:

Reduce computational cost: fewer features → faster model training.

Visualisation: reduce to 2D or 3D to plot and explore data structure.

Remove noise: low-variance components often represent noise, dropping them helps.

Fix multicollinearity: correlated features become uncorrelated principal components.

Before KNN: reduces the curse of dimensionality.

WHEN NOT TO USE PCA:

When interpretability matters — PCA components are combinations of ALL original features.

Individual feature meaning is lost. You cannot say "component 1 represents age."

For tree-based algorithms (Random Forest, XGBoost) — they handle feature selection naturally.

When features are already few and uncorrelated.

 **Remember:** Always standardise features BEFORE applying PCA. PCA is variance-based — features with larger scales will dominate the components without standardisation.

Q11. What is a neural network? What is a hidden layer?

Frequently Asked

A neural network is a machine learning model organised as layers of connected nodes (neurons) that transform input data into predictions through learnable operations.

STRUCTURE:

Input Layer:

One node per input feature. No computation — just receives raw input.

Hidden Layers:

One or more intermediate layers between input and output.

Each layer: applies a linear transformation (weighted sum) + nonlinear activation function.

This is where the model learns complex representations of the data.

Hidden because their values are not directly observed — not input, not output.

Depth = number of hidden layers. Deep network = many hidden layers.

Output Layer:

One node per class (classification) or one node (regression).

Activation depends on task: softmax for multi-class, sigmoid for binary, linear for regression.

WHAT HIDDEN LAYERS DO:

Layer 1: might learn edges and textures in images, basic patterns in text.

Layer 2: might learn shapes from combinations of edges, phrases from words.

Layer 3: might learn object parts, semantic concepts.

Each layer builds more abstract representations from the previous layer.

This hierarchical feature learning is why deep networks are powerful.

WITHOUT HIDDEN LAYERS:

Network = linear model. Cannot learn non-linear relationships.

Even stacking many linear layers without activation = still one linear layer.

Hidden layers + activation functions = ability to learn any complex function.

UNIVERSAL APPROXIMATION THEOREM:

A network with just ONE hidden layer (with enough neurons) can approximate any continuous function. More layers often achieve the same with fewer neurons.

 **Tip:** In interviews: mention that the number of hidden layers = depth, and depth is what makes 'deep learning' deep. Also mention that activation functions between layers are what enable non-linear learning.

Section 3 — Deep Learning Essentials

Q12. What is the difference between BERT and GPT? Encoder vs Decoder?

Frequently Asked

Both are Transformer-based models but designed for different tasks using different parts of the Transformer architecture.

GPT (Generative Pre-trained Transformer) — DECODER ONLY:

Architecture: Transformer decoder stack.

Training: predict the NEXT token given all previous tokens.

"The cat sat on the ____" → predicts "mat"

Attention: CAUSAL (masked) self-attention — each token can only attend to PREVIOUS tokens.

Cannot look ahead — designed for left-to-right generation.

Best for: TEXT GENERATION — completing sequences, answering questions, writing code.

Examples: GPT-4, Claude, LLaMA, Gemini.

BERT (Bidirectional Encoder Representations from Transformers) — ENCODER ONLY:

Architecture: Transformer encoder stack.

Training: MASKED language modelling — predict hidden (masked) tokens.

"The cat [MASK] on the mat" → predicts "sat"

Attention: FULL bidirectional — every token attends to ALL other tokens.

Can see both left and right context simultaneously.

Best for: TEXT UNDERSTANDING — classification, NER, sentiment, question answering.

Examples: BERT, RoBERTa, DistilBERT.

ENCODER-DECODER (Seq2Seq):

Both encoder and decoder stacks.

Encoder reads full input (bidirectional). Decoder generates output autoregressively.

Best for: TRANSLATION, SUMMARISATION, tasks with distinct input and output format.

Examples: T5, BART, original Transformer.

FOR RAG — YOU USE BOTH:

Embedding models (document retrieval) are often ENCODER-based.

Need full bidirectional context to create rich chunk representations.

Generation models (LLM that answers) are DECODER-based — generate token by token.

★ **Key Insight:** Simple rule to remember: GPT/Claude/LLaMA = decoder = generates text. BERT/embedding models = encoder = understands and represents text. Your RAG system uses both: encoder for embeddings, decoder for answering.

⚠ **Interview trap:** People say 'BERT is better than GPT.' They are not competing — they serve different purposes. BERT is better for classification tasks. GPT is better for generation tasks. Comparing them is like comparing a hammer and a screwdriver.

Q13. What is a CNN? What is it good for and why?

Frequently Asked

CNN (Convolutional Neural Network) is a specialised neural network architecture designed for processing data with spatial structure — primarily images.

THE CORE IDEA — CONVOLUTION:

Instead of connecting every pixel to every neuron (very expensive and not scalable), a CNN uses a small filter (kernel) that SLIDES across the image.

At each position it computes a dot product between the filter and the local patch.

The same filter is applied everywhere — this is called weight sharing.

WHY WEIGHT SHARING IS POWERFUL:

A filter that detects vertical edges works the same at the top as the bottom of the image.

No separate parameters needed for "vertical edge at position (10,20)" and at "(100,200)."

Massively fewer parameters than fully connected networks for image tasks.

CNN BUILDING BLOCKS:

Convolutional layer: applies N filters, outputs N feature maps showing where patterns occur.

ReLU activation: non-linearity applied after each convolution.

Pooling layer (Max Pool): downsamples — takes max value in each region.

Reduces spatial size and provides some spatial invariance.

Fully connected layer: final layers for classification after spatial features are extracted.

WHAT CNN LAYERS LEARN (hierarchically):

Early layers: edges, corners, colour gradients — low-level features.

Middle layers: textures, patterns, simple shapes.

Late layers: object parts, faces, specific objects — high-level semantic features.

USE CASES:

Image classification, object detection, facial recognition, medical image analysis.

Also used for: audio spectrograms, time series with local patterns.

NOT GOOD FOR:

Sequential text data — no temporal structure understanding.

That is why RNNs and Transformers are used for text instead of CNNs.

★ **Key Insight:** CNN key ideas: (1) Local connectivity — small filters on local patches. (2) Weight sharing — same filter slides across entire image. (3) Hierarchical features — low-level to high-level. These three ideas make CNNs efficient and powerful for spatial data.